

Unidad III

Clase 4

Diccionario y Expresión

**Diccionario: clave, valor y
diseño modular**

*Entender cómo expresar un TDA en Java
manteniendo modularidad.*



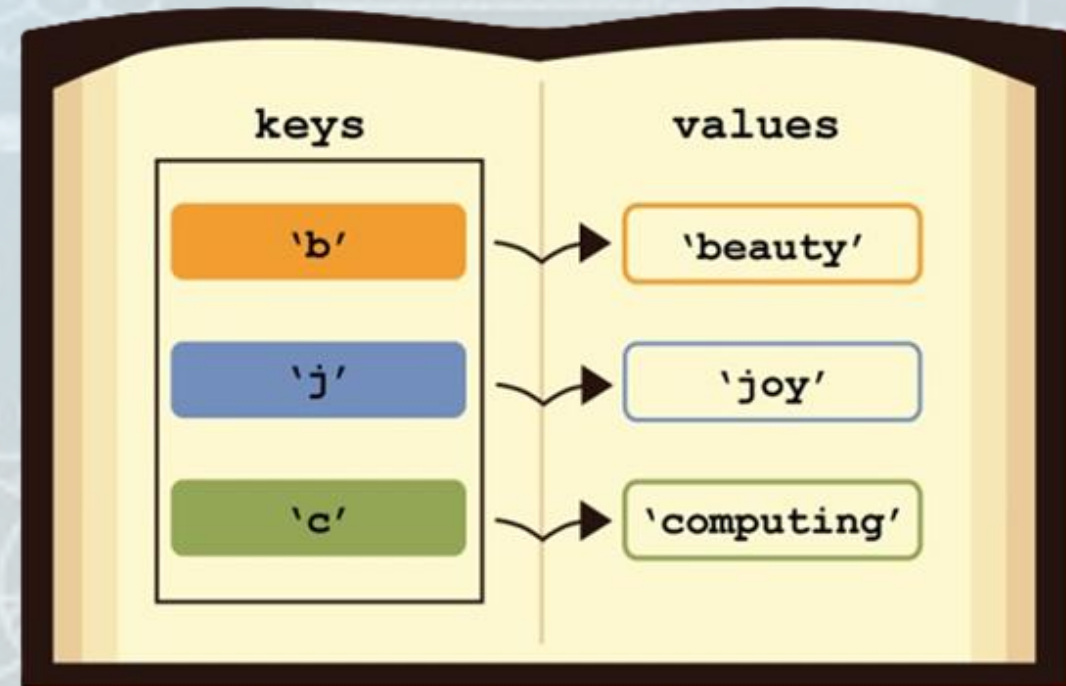
La estructura de datos diccionario se caracteriza porque cada valor ingresa a la estructura asociado a una clave, y estas claves existen siempre que tengan valor asociado y son únicas.

DiccionarioSimple

cada clave puede tener asociado un único valor

DiccionarioMultiple

cada clave tiene asociado un conjunto de valores



TDA DICcionario SIMPLE



Un diccionario simple es un TDA que almacena pares clave-valor, donde cada clave está asociada a un único valor.

OPERACIONES BÁSICAS

- initialize() : crea el diccionario vacío
- put(key, value) : agrega una asociación clave-valor (reemplaza si la clave ya existe)
- remove(key) : elimina el par asociado a la clave
- get(key) : recupera el valor asociado a la clave
- containsKey(key) : indica si la clave existe
- keys() : devuelve el conjunto de claves almacenadas
- isEmpty() : indica si el diccionario está vacío



PRINCIPIO

- KBA (Key-Based Access)
Acceso por clave



TDA DICCIONARIO SIMPLE



`initialize()` → void

`put(key, value)` → void

`remove(key)` → void

Pre: la clave existe

`get(key)` → valor

Pre: la clave existe

`containsKey(key)` → boolean

`keys()` → Conjunto de claves

`isEmpty()` → boolean

8	7	3	95	8	15	7	6					
1	9	8	3	7	4	2	11					
0	1	2	3	4	5	6	7	8				

elementos

índice

Elemento

- clave
- valor

valor

16

clave

5

TDA DICIONARIO SIMPLE



```
public class DiccionarioSimple {

    class Elemento {
        int clave;
        int valor;
    }

    private Elemento[] elementos;
    private int cant;

    public DiccionarioSimple() {
        elementos = new Elemento[100];
        cant = 0;
    }

    public void put(int clave, int valor) {
        int pos = clave2Indice(clave);

        if (pos == -1) {
            pos = cant;
            elementos[pos] = new Elemento();
            elementos[pos].clave = clave;
            cant++;
        }

        elementos[pos].valor = valor;
    }

    public void remove(int clave) {
        int pos = clave2Indice(clave);

        if (pos != -1) {
            elementos[pos] = elementos[cant - 1];
            cant--;
        }
    }
}
```

```
    public int get(int clave) {
        int pos = clave2Indice(clave);
        return elementos[pos].valor;
    }

    public boolean containsKey(int clave) {
        return clave2Indice(clave) != -1;
    }

    public boolean isEmpty() {
        return cant == 0;
    }

    private int clave2Indice(int clave) {
        int i = cant - 1;

        while (i >= 0 && elementos[i].clave != clave) {
            i--;
        }

        return i;
    }
}
```

```
public class Main {
    public static void main(String[] args) {

        DiccionarioSimple dic = new DiccionarioSimple();

        dic.put(3, 95);
        dic.put(8, 21);
        dic.put(3, 7); // reemplaza

        System.out.println(dic.get(3)); // imprime 7

        dic.remove(8);

        System.out.println(dic.containsKey(8)); // false
    }
}
```

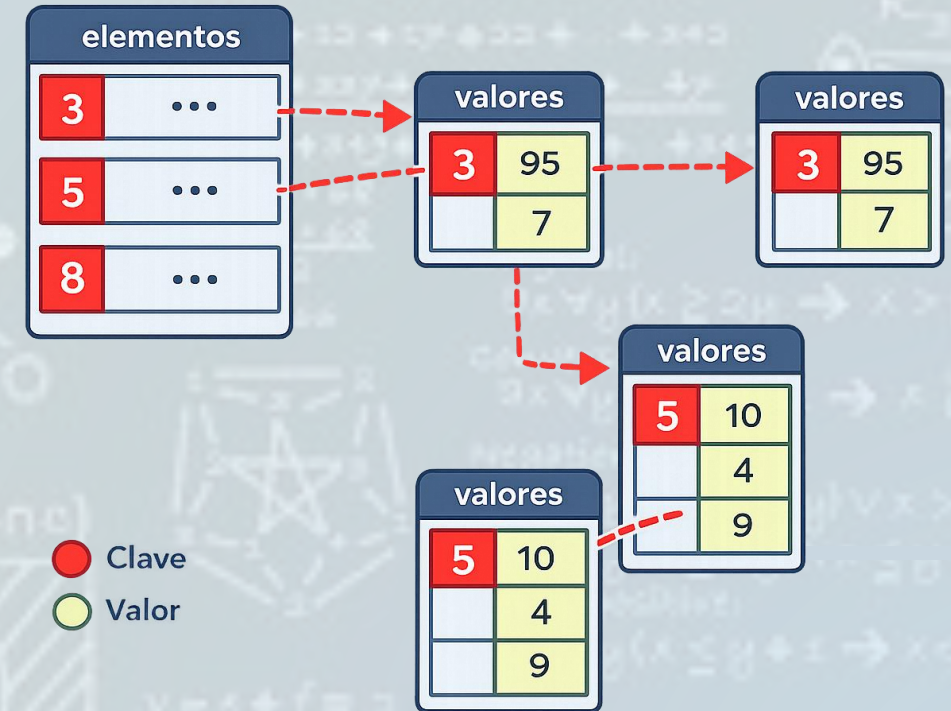
TDA DICCIONARIO MULTIPLE



Un diccionario múltiple es un TDA que almacena pares clave-valor, donde una misma clave puede estar asociada a varios valores distintos.

OPERACIONES BÁSICAS

- initialize() : crea el diccionario vacío
- put(key, value) : agrega un valor a la clave (no reemplaza valores existentes)
- remove(key) : elimina la clave y todos sus valores asociados
- removeValue(key, value) : elimina un valor específico de la clave
- get(key) : devuelve todos los valores asociados a la clave
- containsKey(key) : indica si la clave existe
- keys() : devuelve el conjunto de claves almacenadas
- Empty() : indica si el diccionario está vacío



PRINCIPIO

- KBA (Key-Based Access)
Acceso por clave



`initialize() → void`

`put(key, value) → void`

`remove(key) → void`

Pre: la clave existe

`removeValue(key, value) → void`

Pre: la clave y el valor existen

`get(key) → valor`

Pre: la clave existe

`containsKey(key) → boolean`

`keys() → Conjunto de claves`

`isEmpty() → boolean`

UADE

TDA DICcionario MULTIPLE



elementos

valores

3	8	7	3	95					
	0	1	2	3					
7	4	45	22	3	8				
	0	1	2	3	4				
5	8	31	52						
	0	1	2						

valor

16

clave

5

Elemento

- Clave int
- Valor int[]

```
public class DiccionarioMultiple {
```

```
    class Elemento {
        int clave;
        int[] valores;
        int cantValores;
    }
```

```
    private Elemento[] elementos;
    private int cantClaves;
```

```
    public DiccionarioMultiple() {
        elementos = new Elemento[100];
        cantClaves = 0;
    }
```

```
    public void initialize() {
        elementos = new Elemento[100];
        cantClaves = 0;
    }
```

```
    public void put(int clave, int valor) {
        int posC = clave2Indice(clave);
```

```
        if (posC == -1) {
            posC = cantClaves;
            elementos[posC] = new Elemento();
            elementos[posC].clave = clave;
            elementos[posC].valores = new int[100];
            elementos[posC].cantValores = 0;
            cantClaves++;
        }
```

```
        Elemento e = elementos[posC];
        int posV = valor2Indice(e, valor);
```

```
        if (posV == -1) {
            e.valores[e.cantValores] = valor;
            e.cantValores++;
        }
```

```
    }
```

```
    public void remove(int clave) {
        int pos = clave2Indice(clave);
```

```
        if (pos != -1) {
            elementos[pos] = elementos[cantClaves - 1];
            cantClaves--;
        }
```

```
    }
```

```
    public void removeValue(int clave, int valor) {
        int posC = clave2Indice(clave);
```

```
        if (posC != -1) {
            Elemento e = elementos[posC];
            int posV = valor2Indice(e, valor);
```

```
            if (posV != -1) {
                e.valores[posV] = e.valores[e.cantValores - 1];
                e.cantValores--;
            }
```

```
            if (e.cantValores == 0) {
                remove(clave);
            }
```

```
        }
```

```
    }
```

```
    public int[] get(int clave) {
        int pos = clave2Indice(clave);
```

```
        if (pos == -1) {
            return new int[0];
        }
```

```
        Elemento e = elementos[pos];
        int[] resultado = new int[e.cantValores];
```

```
        for (int i = 0; i < e.cantValores; i++) {
            resultado[i] = e.valores[i];
        }
```

```
        return resultado;
```

```
    }
```

TDA DICcionario MULTIPLE



```
    public boolean containsKey(int clave) {
        return clave2Indice(clave) != -1;
    }
```

```
    public int[] keys() {
        int[] resultado = new int[cantClaves];
```

```
        for (int i = 0; i < cantClaves; i++) {
            resultado[i] = elementos[i].clave;
        }
```

```
        return resultado;
```

```
    }
```

```
    public boolean isEmpty() {
        return cantClaves == 0;
    }
```

```
    private int clave2Indice(int clave) {
        int i = cantClaves - 1;
```

```

while (i >= 0 && elementos[i].clave != clave) {
    i--;
}

return i;
}

private int valor2Indice(Elemento e, int valor) {
    int i = e.cantValores - 1;

    while (i >= 0 && e.valores[i] != valor) {
        i--;
    }

    return i;
}

public void mostrar() {
    if (cantClaves == 0) {
        System.out.println("El diccionario multiple esta vacio.");
        return;
    }

    for (int i = 0; i < cantClaves; i++) {
        System.out.print("Clave " + elementos[i].clave + " -> { ");

        for (int j = 0; j < elementos[i].cantValores; j++) {
            System.out.print(elementos[i].valores[j]);

            if (j < elementos[i].cantValores - 1) {
                System.out.print(", ");
            }
        }

        System.out.println(" }");
    }
}
}

```

```

public class Main {
    public static void main(String[] args) {

        DiccionarioMultiple dic = new DiccionarioMultiple();

        dic.put(3, 95);
        dic.put(3, 7);
        dic.put(3, 8);

        dic.put(5, 10);
        dic.put(5, 4);
        dic.put(5, 9);

        dic.put(7, 45);
        dic.put(7, 22);

        System.out.println("Estado inicial del diccionario:");
        dic.mostrar();

        System.out.println("\nAgregar valor repetido a clave 3:");
        dic.put(3, 95); // no lo agrega porque ya existe
        dic.mostrar();

        System.out.println("\nValores asociados a la clave 5:");
        int[] valores = dic.get(5);
        for (int i = 0; i < valores.length; i++) {
            System.out.print(valores[i] + " ");
        }

        System.out.println("\n\nEliminar valor 4 de la clave 5:");
        dic.removeValue(5, 4);
        dic.mostrar();

        System.out.println("\n\nEliminar clave 7 completa:");
        dic.remove(7);
        dic.mostrar();

        System.out.println("\nClaves almacenadas:");
        int[] claves = dic.keys();
        for (int i = 0; i < claves.length; i++) {
            System.out.print(claves[i] + " ");
        }

        System.out.println("\n\nExiste la clave 3? " + dic.containsKey(3));
        System.out.println("Existe la clave 7? " + dic.containsKey(7));
        System.out.println("El diccionario esta vacio? " + dic.isEmpty());
    }
}

```

TDA DICcionario MULTIPLE



ANÁLISIS DE COSTOS



Cuando queremos comparar distintas implementaciones de una estructura de datos, necesitamos medir su eficiencia.

*Para ello utilizamos el **análisis de complejidad**, que evalúa el costo de ejecución de los algoritmos.*

Tipos

Análisis de complejidad temporal

Mide la cantidad de operaciones que realiza un algoritmo.

Permite estimar cuánto tiempo tarda en ejecutarse según la cantidad de datos. Se utiliza para comparar la eficiencia de distintas implementaciones.

Análisis de complejidad de espacio

Mide la cantidad de memoria que utiliza un algoritmo.

Permite evaluar cuánto espacio adicional requiere la estructura. Se utiliza cuando la memoria disponible es un factor crítico.



Costo constante (C)

El tiempo de ejecución no depende de la cantidad de elementos. Siempre se ejecuta la misma cantidad de operaciones. Es el mejor caso posible.

Costo lineal (L)

El tiempo de ejecución depende de la cantidad de elementos. Se recorren los datos una sola vez. El costo crece proporcionalmente.

Costo polinómico (P)

El tiempo de ejecución es mayor que lineal. Se realizan múltiples recorridos o comparaciones anidadas. El costo crece rápidamente.

Análisis de la Pila



Operación	PilaTI	PilaTF	Explicación
Inicializar	C	C	Solo asigna valores iniciales
Apilar	L	C	Inserta en posición conocida
Desapilar	L	C	Quita el último elemento
Tope	C	C	Acceso directo al índice
PilaVacía	C	C	Comparación simple

PilaTI → Pila con Tope Inicial (siempre busca la primera posición no guarda.

Tope comienza en 0 y crece hacia arriba)

PilaTF → Pila con Tope Final (tope comienza en el final y decrece. El tope se accede directamente por el índice)

Todas las operaciones de la pila tienen **costo constante**, porque no recorren la estructura.

Actividad

Analisis de costos

Analizar el costo de complejidad temporal de las siguientes estructuras:

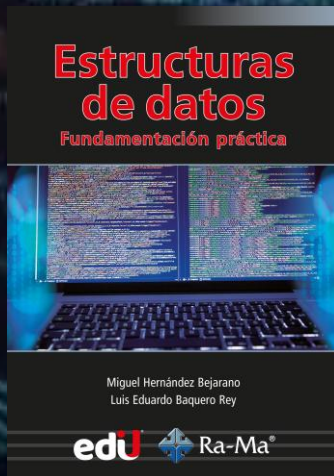
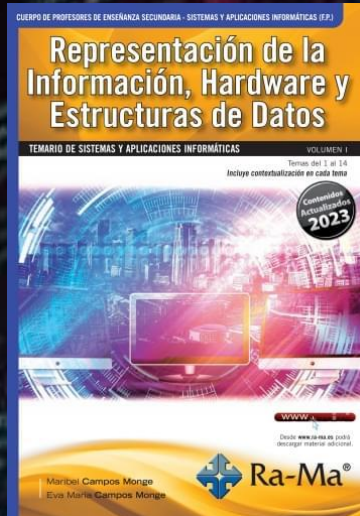
- Cola con prioridad (implementación 1: arreglo paralelo)
- Cola con prioridad (implementación 2: objetos)
- Cola con prioridad (implementación 3: tres colas)
- Conjunto
- Diccionario simple
- Diccionario múltiple

Clasificar cada operación como:

C → constante

L → lineal

P → polinómico



BIBLIOGRAFIA



Cuadrado Estrebou, María Fernanda Trutner, Guillermo Hernán. Revisión: J.Tempio.
Programación II. Apuntes de cátedra.



PROXIMO ENCUENTRO



Clase 5

Combinar estructuras para resolver problemas reales

Resolución de problemas combinando Pila, Cola, Conjunto y Diccionario..